

Assignment 5: Developing a Class Based Ride Sharing System

Milan Bista

2025 Spring – Advanced Programming Languages (MSCS-632-M51) – Full Term

Instructores: Vanessa Cooper / machica McClain

GitHub Link: <https://github.com/mbista25742/MSCS-632-M51/tree/main/assignment5>

1. Introduction

This report details the implementation of a Ride Sharing System using Object-Oriented Programming (OOP) principles in two distinct programming languages: Smalltalk and C++. The project aims to demonstrate core OOP concepts, namely encapsulation, inheritance, and polymorphism, through the development of a functional ride sharing application.

2. Application Requirements

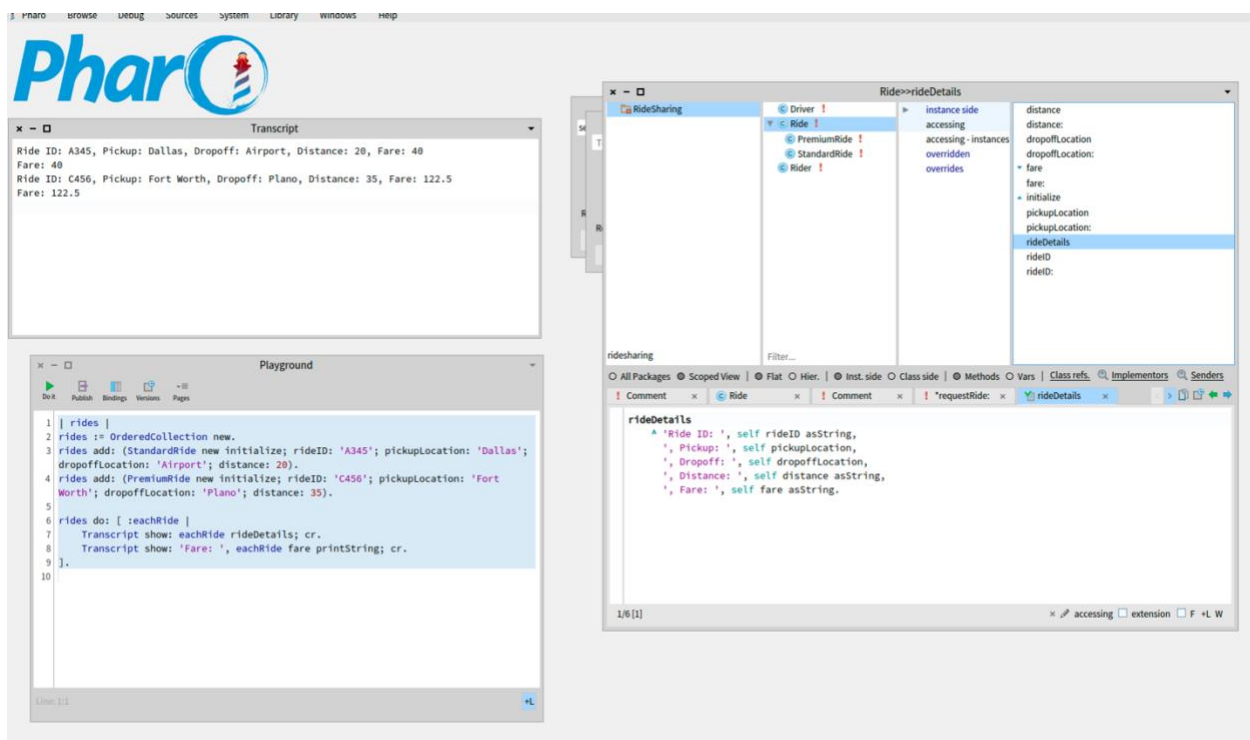
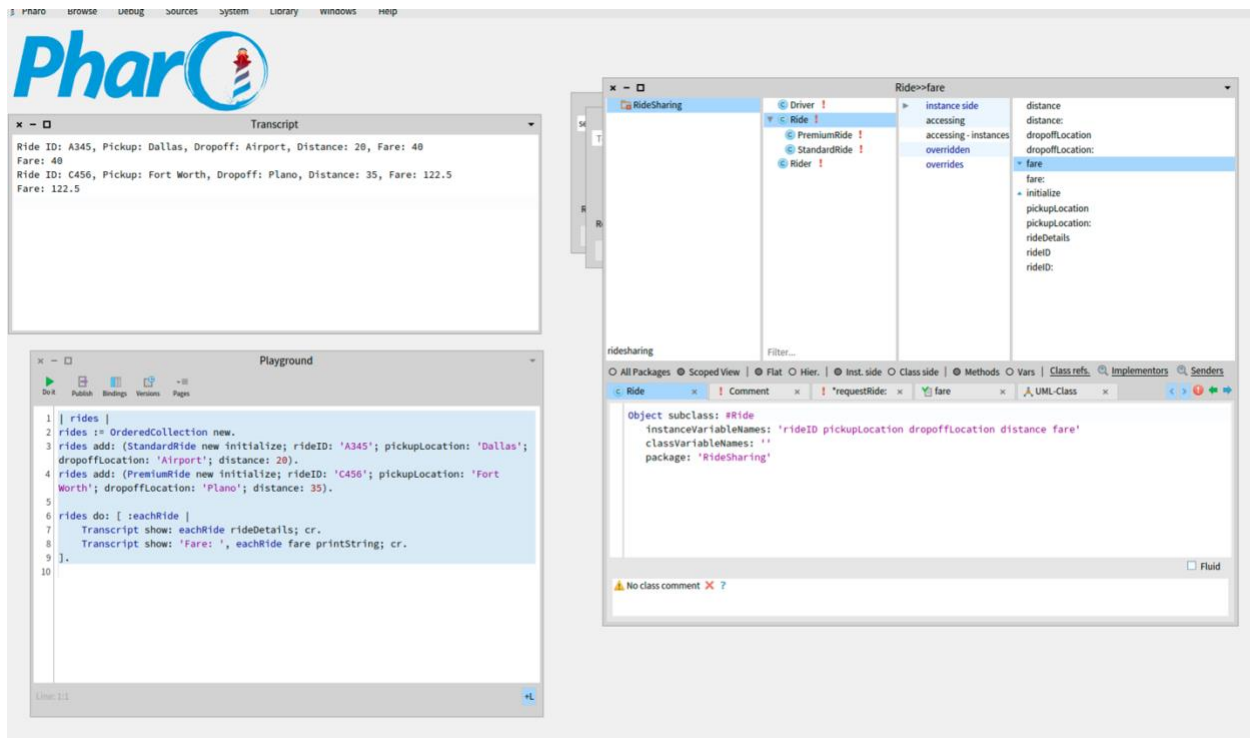
The Ride Sharing System was designed to meet the following requirements:

- Ride Class: A base class to hold core ride information such as rideID, pickup location, dropoff location, distance, and fare.
- Specific Ride Subclasses: At least two derived classes (e.g., StandardRide and PremiumRide), each overriding the fare calculation method to reflect different pricing models.
- Driver Class: A class to manage driver information, including assigned rides.
- Rider Class: A class to manage rider information and track requested rides.
- System Functionality: Demonstration of polymorphism through storing and processing rides of different types.

3. Implementation Overview

(a) Smalltalk Implementation

The Smalltalk implementation utilized classes to represent the Ride, Driver, and Rider entities. The Ride class served as the base class, with StandardRide and PremiumRide as subclasses, each implementing its own fare calculation logic. The Driver and Rider classes managed their respective attributes and interactions.



(b) C++ Implementation

The C++ implementation followed a similar structure, employing classes and inheritance to model the system. The Ride class was defined as an abstract base class with virtual

fare calculation methods, which were then overridden in the StandardRide and PremiumRide subclasses.

```
rideshare.cpp x
1 #include <iostream>
2 #include <string>
3 #include <vector>
4
5 class Ride {
6 public:
7     Ride(int id, const std::string& pickup, const std::string& dropoff, double distance) :
8         rideID(id), pickupLocation(pickup), dropoffLocation(dropoff), calculateFare();
9
10    }
11
12    virtual ~Ride() {} // Virtual destructor for polymorphism
13
14    int getRideID() const { return rideID; }
15    std::string getPickupLocation() const { return pickupLocation; }
16    std::string getDropoffLocation() const { return dropoffLocation; }
17    double getDistance() const { return distance; }
18    double getFare() const { return fare; }
19
20    virtual void calculateFare() {
21        fare = distance * 10.0; // Default fare calculation
22    }
23
24    virtual std::string rideDetails() const {
25        return "Ride ID: " + std::to_string(rideID) +
26            ", Pickup: " + pickupLocation +
27            ", Dropoff: " + dropoffLocation +
28            ", Distance: " + std::to_string(distance) +
29            ", Fare: " + std::to_string(fare);
30    }
31
32 protected:
33     double distance; // Changed to protected
34     double fare;
```

```
rideshare.cpp x
96 class Rider {
100 void requestRide(Ride* ride) {
102 }
103
104 void viewRides() const {
105     for (const auto& ride : requestedRides) {
106         std::cout << ride->rideDetails() << std::endl;
107     }
108 }
109
110 private:
111     int riderID;
112     std::string name;
113     std::vector<Ride*> requestedRides;
114 };
115
116 int main() {
117     std::vector<Ride*> rides;
118     rides.push_back(new StandardRide(1, "Downtown", "Airport", 10));
119     rides.push_back(new PremiumRide(2, "Fort Worth", "Plano", 35));
120     rides.push_back(new Ride(3, "Home", "Office", 5));
121
122     for (const auto& ride : rides) {
123         std::cout << ride->rideDetails() << std::endl;
124         std::cout << "Fare: " << ride->getFare() << std::endl;
125     }
126
127     // Clean up memory (important!)
128     for (auto ride : rides) {
129         delete ride;
130     }
131
132     return 0;
```

```
assignment5 -- -zsh -- 155x49
(base) milanbista@Milans-MacBook-Pro assignment5 % c++ -std=c++11 rideshare.cpp -o rideshare
(base) milanbista@Milans-MacBook-Pro assignment5 % ./rideshare
Ride ID: 1, Pickup: Downtown, Dropoff: Airport, Distance: 10.000000, Fare: 100.000000
Fare: 100
Ride ID: 2, Pickup: Fort Worth, Dropoff: Plano, Distance: 35.000000, Fare: 350.000000
Fare: 350
Ride ID: 3, Pickup: Home, Dropoff: Office, Distance: 5.000000, Fare: 50.000000
Fare: 50
(base) milanbista@Milans-MacBook-Pro assignment5 %
```

4. OOP Principles Demonstrated

- Encapsulation: The Driver and Rider classes encapsulated their attributes and provided controlled access through methods, ensuring data integrity.
- Inheritance: The StandardRide and PremiumRide classes inherited from the Ride base class, demonstrating code reuse and a hierarchical class structure.
- Polymorphism: The system showcased polymorphism by storing and processing rides of different types through a common Ride interface, allowing for dynamic method invocation.

7. Conclusion

This project successfully implemented a Ride Sharing System in both Smalltalk and C++, demonstrating the application of core OOP principles. The implementations effectively modeled the system's entities and their interactions, showcasing the flexibility and power of object-oriented programming.

References

- Sebesta, R. W. (2018). Concepts of Programming Languages (12th ed.). Pearson Education (US). <https://reader2.yuzu.com/books/9780135102251>